

CO5-AT3-CODING CHALLENGE

Q1. Library Management System

```
#include <iostream>

#include <string>

using namespace std;

class Book {

private:

    int id;

    string title, author, category;

    bool available;

public:

    static int totalIssued;

    void input() {

        cout << "Book ID: ";

        cin >> id;

        cin.ignore();

        cout << "Title: ";

        getline(cin, title);

        cout << "Author: ";

        getline(cin, author);

        cout << "Category: ";

        getline(cin, category);

        available = true;

    }

    int getId() {

        return id;

    }

    string getCategory() {

        return category;

    }

    bool isAvailable() {

        return available;

    }

}
```

```

    }
    void issueBook() {
        if (available) {
            available = false;
            totalIssued++;
            cout << "Book issued successfully\n";
        } else {
            cout << "Book is not available\n";
        }
    }
}

void returnBook() {
    if (!available) {
        available = true;
        totalIssued--;
        cout << "Book returned successfully\n";
    } else {
        cout << "Book was not issued\n";
    }
}

void display() {
    cout << id << " | " << title << " | "
        << author << " | " << category
        << " | " << (available ? "Available" : "Issued") << endl;
}

};

int Book::totalIssued = 0;

int main() {
    int n, choice, id;
    Book books[1000];
    cout << "Number of books: ";
    cin >> n;
    for (int i = 0; i < n; i++)

```

```

        books[i].input();
do {
    cout << "\n1.Issue 2.Return 3.Display Category 4.Total Issued 5.Exit\n";
    cin >> choice;
    if (choice == 1 || choice == 2) {
        cout << "Enter Book ID: ";
        cin >> id;
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (books[i].getId() == id) {
                found = true;
                if (choice == 1)
                    books[i].issueBook();
                else
                    books[i].returnBook();
                break;
            }
        }
        if (!found)
            cout << "Invalid Book ID\n";
    }
    else if (choice == 3) {
        string category;
        cin.ignore();
        cout << "Enter Category: ";
        getline(cin, category);
        for (int i = 0; i < n; i++) {
            if (books[i].getCategory() == category &&
                books[i].isAvailable())
                books[i].display();
        }
    }
}

```

```

        else if (choice == 4)

            cout << "Total Issued: " << Book::totalIssued << endl;

    } while (choice != 5);

    return 0;

}

```

The screenshot shows a C++ IDE with a file named `main.cpp`. The code implements a book management system with the following logic:

- Line 97: `else` block for returning a book.
- Line 98: `books[i].returnBook();`
- Line 99: `break;`
- Line 100: `}`
- Line 101: `}`
- Line 102: `if (!found)` block for invalid book ID.
- Line 103: `cout << "Invalid Book ID\n";`
- Line 104: `}`
- Line 105: `else if (choice == 3) {` block for adding a new book.
- Line 106: `string category;`
- Line 107: `cin.ignore();`
- Line 108: `cout << "Enter Category: ";`
- Line 109: `getline(cin, category);`
- Line 110: `}`
- Line 111: `for (int i = 0; i < n; i++) {` loop to find a book by category.
- Line 112: `if (books[i].getCategory() == category &&`
- Line 113: `books[i].isAvailable())`
- Line 114: `books[i].display();`
- Line 115: `}`
- Line 116: `}`
- Line 117: `else if (choice == 4)` block for displaying total issued books.
- Line 118: `cout << "Total Issued: " << Book::totalIssued << endl;`
- Line 119: `}`

The output window shows the following input and output:

```

Title: hary potter
Author: jk rowling
Category: fiction
Book ID: 102
Title: physics
Author: hc verma
Category: reference

```

Q2. Parking System

```

#include <iostream>

using namespace std;

class ParkingSlot {
private:
    int vehicleNumber;

    int entryTime;

    bool occupied;
public:
    static int occupiedSlots;

    ParkingSlot() {
        occupied = false;
    }

    void park(int vehicle, int time) {
        if (!occupied) {
            vehicleNumber = vehicle;

```

```

        entryTime = time;
        occupied = true;
        occupiedSlots++;
        cout << "Vehicle parked successfully\n";
    } else {
        cout << "Slot already occupied\n";
    }
}

void remove(int exitTime) {
    if (occupied) {
        int duration = exitTime - entryTime;
        if (duration < 0) {
            cout << "Invalid time\n";
            return;
        }
        int charge = duration * 2;
        cout << "Parking duration: " << duration << " minutes\n";
        cout << "Charge: Rs." << charge << endl;
        occupied = false;
        occupiedSlots--;
    } else {
        cout << "Slot is empty\n";
    }
}

bool isOccupied() {
    return occupied;
}

};

int ParkingSlot::occupiedSlots = 0;

int main() {
    int n;
    cout << "Enter number of parking slots: ";

```

```

cin >> n;

ParkingSlot slots[500];

int choice;

do {
    cout << "\n1.Park 2.Exit 3.Status 4.Exit Program\n";
    cin >> choice;
    int slot, vehicle, time;
    if (choice == 1) {
        cout << "Slot number, Vehicle number, Entry time: ";
        cin >> slot >> vehicle >> time;
        if (slot >= 0 && slot < n)
            slots[slot].park(vehicle, time);
        else
            cout << "Invalid slot\n";
    }
    else if (choice == 2) {
        cout << "Slot number and Exit time: ";
        cin >> slot >> time;

        if (slot >= 0 && slot < n)
            slots[slot].remove(time);
        else
            cout << "Invalid slot\n";
    }
    else if (choice == 3)
        cout << "Occupied slots: "
            << ParkingSlot::occupiedSlots << endl;
} while (choice != 4);

return 0;

```

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  class Parking {
5  public:
6      static int charge;
7
8      void calculate(int minutes) {
9          cout << "Charge: " << minutes * charge;
10     }
11 };
12
13 int Parking::charge = 2;
14
15 int main() {
16     int time;
17     cin >> time;
18
19     Parking p;
20     p.calculate(time);
21 }
```

input

120
Charge: 240

Q3. Employee Salary System

```
#include <iostream>
#include <string>
using namespace std;
class Employee {
public:
    string name;
    double salary;
    void input() {
        cin >> name >> salary;
    }
};
int main() {
    int n;
    cin >> n;
    Employee e[100];
    int high = 0;
```

```

for (int i = 0; i < n; i++) {
    e[i].input();

    if (e[i].salary > e[high].salary)
        high = i;
}
cout << "Highest: " << e[high].name
    << " " << e[high].salary;
}

```

```

5  class Employee {
6      public:
7          string name;
8          double salary;
9
10     void input() {
11         cin >> name >> salary;
12     }
13 };
14
15 int main() {
16     int n;
17     cin >> n;
18
19     Employee e[100];
20     int high = 0;
21
22     for (int i = 0; i < n; i++) {
23         e[i].input();
24
25         if (e[i].salary > e[high].salary)
26             high = i;
27     }
28
29     cout << "Highest: " << e[high].name
30         << " " << e[high].salary;
31 }

```

input

```

ravi 3000
priya 5000
anil 4000
Highest: priya 5000

```


Q4. Bank Account System

```
#include <iostream>

using namespace std;

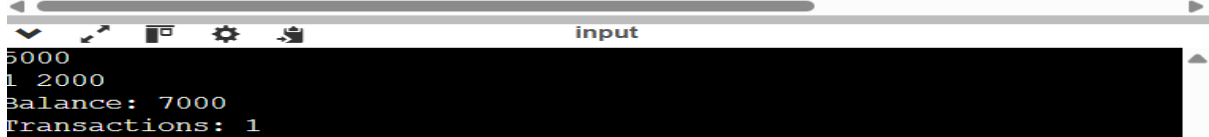
class Account {
public:
    double balance;
    static int transactions;
    void deposit(double amount) {
        balance += amount;
        transactions++;
    }
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            transactions++;
        } else
            cout << "Insufficient balance\n";
    }
};

int Account::transactions = 0;

int main() {
    Account a;
    cin >> a.balance;
    int choice;
    double amount;
    cin >> choice >> amount;
    if (choice == 1)
        a.deposit(amount);
    else
        a.withdraw(amount);
    cout << "Balance: " << a.balance;
    cout << "\nTransactions: " << Account::transactions;
```

```
}
```

```
14 - void withdraw(double amount) {
15 -     if (amount <= balance) {
16 -         balance -= amount;
17 -         transactions++;
18 -     } else
19 -         cout << "Insufficient balance\n";
20 - }
21 - };
22 -
23 - int Account::transactions = 0;
24 -
25 - int main() {
26 -     Account a;
27 -     cin >> a.balance;
28 -
29 -     int choice;
30 -     double amount;
31 -     cin >> choice >> amount;
32 -
33 -     if (choice == 1)
34 -         a.deposit(amount);
35 -     else
36 -         a.withdraw(amount);
37 -
38 -     cout << "Balance: " << a.balance;
39 -     cout << "\nTransactions: " << Account::transactions;
40 - }
```



5000
1 2000
Balance: 7000
Transactions: 1

Q5. E-Commerce Bill System

```
#include <iostream>
```

```
using namespace std;
```

```
class Product {
```

```
public:
```

```
    double price;
```

```
    void input() {
```

```
        cin >> price;
```

```
    }
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    Product p[100];
```

```
    double total = 0, highest = 0;
```

```

for (int i = 0; i < n; i++) {
    p[i].input();
    total += p[i].price;
    if (p[i].price > highest)
        highest = p[i].price;
if (total >= 2000)
    total *= 0.9;
cout << "Bill: " << total;
cout << "\nHighest price: " << highest;
}

```

```

7
8 void input() {
9     cin >> price;
10 }
11 };
12
13 int main() {
14     int n;
15     cin >> n;
16
17     Product p[100];
18     double total = 0, highest = 0;
19
20     for (int i = 0; i < n; i++) {
21         p[i].input();
22         total += p[i].price;
23
24         if (p[i].price > highest)
25             highest = p[i].price;
26     }
27
28     if (total >= 2000)
29         total *= 0.9;
30
31     cout << "Bill: " << total;
32     cout << "\nHighest price: " << highest;
33 }

```

input

```

2000
500
Bill: 3150
Highest price: 2000

```

Q6. Hospital Patient System

```
#include <iostream>
#include <string>
using namespace std;
class Patient {
public:
    string name;
    int age;
    bool critical;
    void input() {
        cin >> name >> age >> critical;
    }
};
int main() {
    int n;
    cin >> n;
    Patient p[100];
    for (int i = 0; i < n; i++)
        p[i].input();
    cout << "Critical patients:\n";
    for (int i = 0; i < n; i++) {
        if (p[i].critical)
            cout << p[i].name << endl;
    }
}
```

```

5 class Patient {
6 public:
7     string name;
8     int age;
9     bool critical;
10
11     void input() {
12         cin >> name >> age >> critical;
13     }
14 };
15
16 int main() {
17     int n;
18     cin >> n;
19
20     Patient p[100];
21
22     for (int i = 0; i < n; i++)
23         p[i].input();
24
25     cout << "Critical patients:\n";
26
27     for (int i = 0; i < n; i++) {
28         if (p[i].critical)
29             cout << p[i].name << endl;
30     }
31 }

```

input

```

3
ravi 25 0
priya 40 1
anil 60 1
Critical patients:
priya
anil

```

Q7. School Marks System

```

#include <iostream>

using namespace std;

class Student {
public:
    int marks, total = 0;

    void input(int m) {
        for (int i = 0; i < m; i++) {

```

```

        cin >> marks;

        total += marks;
    }
}

};

int main() {
    int n, m, topper = 0;

    cin >> n >> m;

    Student s[100];

    for (int i = 0; i < n; i++) {
        s[i].input(m);

        if (s[i].total > s[topper].total)
            topper = i;
    }

    cout << "Topper total: " << s[topper].total;
}

```

The screenshot shows a C++ program in a code editor and its execution output in a terminal window. The code defines a `Student` class with `marks` and `total` attributes, and an `input` method that takes a number `m` and reads `m` marks from the user, adding them to the `total`. The `main` function creates an array of `Student` objects, reads `n` and `m`, and then iterates through the array, calling `input` on each student and updating the `topper` index if a student's total is greater than the current `topper`'s total. Finally, it prints the total of the `topper`.

```

4  class Student {
5  public:
6      int marks, total = 0;
7
8      void input(int m) {
9          for (int i = 0; i < m; i++) {
10             cin >> marks;
11             total += marks;
12         }
13     }
14 };
15
16 int main() {
17     int n, m, topper = 0;
18     cin >> n >> m;
19
20     Student s[100];
21
22     for (int i = 0; i < n; i++) {
23         s[i].input(m);
24
25         if (s[i].total > s[topper].total)
26             topper = i;
27     }
28
29     cout << "Topper total: " << s[topper].total;
30 }

```

The terminal window shows the input and output of the program:

```

2 3
80 90 85
95 90 92
Topper total: 277

```

Q8. Electricity Bill System

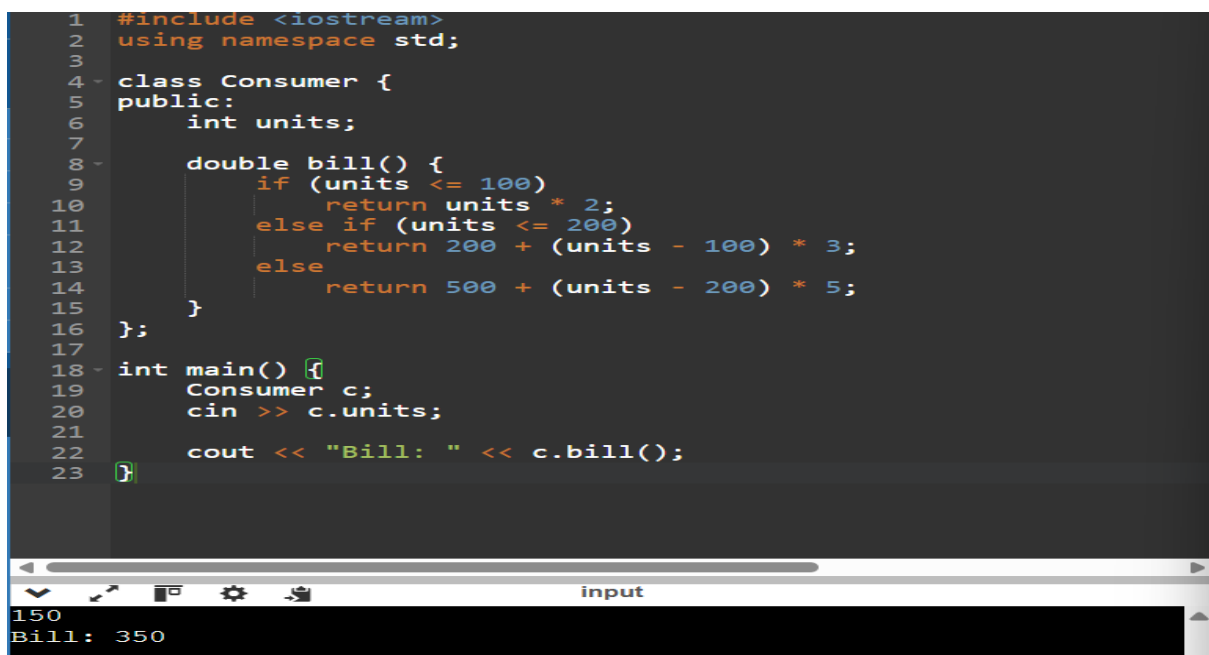
```
#include <iostream>

using namespace std;

class Consumer {
public:
    int units;

    double bill() {
        if (units <= 100)
            return units * 2;
        else if (units <= 200)
            return 200 + (units - 100) * 3;
        else
            return 500 + (units - 200) * 5;
    }
};

int main() {
    Consumer c;
    cin >> c.units;
    cout << "Bill: " << c.bill();
}
```



```
1  #include <iostream>
2  using namespace std;
3
4  class Consumer {
5  public:
6      int units;
7
8      double bill() {
9          if (units <= 100)
10             return units * 2;
11          else if (units <= 200)
12             return 200 + (units - 100) * 3;
13          else
14             return 500 + (units - 200) * 5;
15      }
16  };
17
18  int main() {
19      Consumer c;
20      cin >> c.units;
21
22      cout << "Bill: " << c.bill();
23  }
```

input

150
Bill: 350

Q9. Airline Seat Booking

```
#include <iostream>

using namespace std;

class Flight {
public:
    bool seat[300];

    Flight() {
        for (int i = 0; i < 300; i++)
            seat[i] = false;
    }

    void book(int n) {
        if (!seat[n]) {
            seat[n] = true;
            cout << "Booked";
        } else
            cout << "Already booked";
    }
};

int main() {
    Flight f;
    int seatNo;
    cin >> seatNo;
    f.book(seatNo - 1);
}
```



```
1 #include <iostream>
2 using namespace std;
3
4 class Flight {
5 public:
6     bool seat[300];
7
8     Flight() {
9         for (int i = 0; i < 300; i++)
10             seat[i] = false;
11     }
12
13     void book(int n) {
14         if (!seat[n]) {
15             seat[n] = true;
16             cout << "Booked";
17         } else
18             cout << "Already booked";
19     }
20 };
21
22 int main() {
23     Flight f;
24     int seatNo;
25
26     cin >> seatNo;
27     f.book(seatNo - 1);
28 }
```

input

5

Booked

...Program finished with exit code 0
Press ENTER to exit console.

Q10. Warehouse Inventory System

```
#include <iostream>
```

```
using namespace std;
```

```
class Product {
```

```
public:
```

```
    long long quantity;
```

```
    double price;
```

```
    void input() {
```

```
        cin >> quantity >> price;
```

```
    }
```

```
    double value() {
```

```
        return quantity * price;
```

```
    }
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```

Product p[100];

double total = 0;

for (int i = 0; i < n; i++) {

    p[i].input();

    total += p[i].value();

}

cout << "Total inventory value: " << total;

}

```

```

main.cpp
5 public:
6     long long quantity;
7     double price;
8
9     void input() {
10         cin >> quantity >> price;
11     }
12
13     double value() {
14         return quantity * price;
15     }
16 };
17
18 int main() {
19     int n;
20     cin >> n;
21
22     Product p[100];
23     double total = 0;
24
25     for (int i = 0; i < n; i++) {
26         p[i].input();
27         total += p[i].value();
28     }
29
30     cout << "Total inventory value: " << total;
31 }

```

input

```

2
5 50000
10 500
Total inventory value: 255000

```